

University Regulation RAG Assistant

Project Report

Department of Computer Science & Engineering

Vignan University

September 17, 2026

1 Abstract

University Regulation RAG Assistant is an intelligent, AI-powered academic query system built using Retrieval-Augmented Generation (RAG). The system allows students and faculty to interactively query academic regulation handbooks, examination guidelines, attendance policies, and student rulebooks in natural language and receive accurate, grounded, source-cited responses. It combines vector similarity search using FAISS or ChromaDB with language model answer generation, wrapped in a simple web interface. The system ensures zero hallucinations by enforcing document grounding and explicitly notifying users when a policy query cannot be found in the uploaded documents.

2 Introduction

Academic institutions maintain complex regulation handbooks covering hundreds of rules across semesters, attendance condonation, re-evaluation procedures, credit transfers, and disciplinary policies. Students often struggle to find specific policy details quickly, especially during examination periods and registration deadlines. Traditional PDF keyword search is inflexible and fails when queries do not match exact policy phrasing.

University Regulation RAG Assistant solves this problem by ingesting regulation PDFs and enabling natural language Q&A over them. A student can ask questions like:

- "What is the minimum attendance required to write end exams?"
- "What are the rules and fees for re-evaluation?"
- "How many backlogs are allowed for placement registration?"
- "What is the policy for condonation of attendance?"

The system understands these queries, retrieves relevant regulation sections, and generates a clear, cited answer in real time.

3 Problem Statement

Existing university portals and standard search tools lack the ability to:

- Answer contextual, multi-faceted regulation queries in natural language.
- Return grounded responses with source citations and exact page numbers from rulebooks.
- Prevent AI hallucinations when answering critical academic policy questions.
- Indicate clearly when a specific regulation is missing from the database.

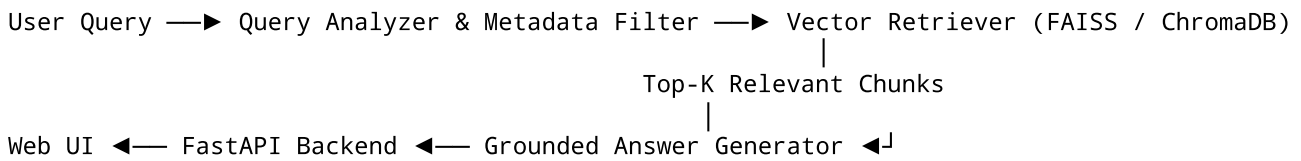
University Regulation RAG Assistant addresses all of the above with a structured, modular RAG pipeline.

4 Objectives

- Build a Retrieval-Augmented Generation (RAG) system tailored to university regulations.
- Parse and index multi-page regulation PDFs automatically.
- Implement recursive text chunking and vector embeddings.
- Store and retrieve document vectors efficiently using FAISS or ChromaDB.
- Support metadata filtering based on document categories and page numbers.
- Generate grounded answers with clear source document and page references.
- Implement fallback mechanisms when answers cannot be found in the documents.

5 System Architecture

The system follows a standard RAG architecture with grounding guardrails:



6 Modules and Components

6.1 PDF Parser (parser.py)

Extracts text from uploaded regulation PDFs using PyPDF or pdfplumber. It identifies section headers, clause numbers, policy lines, and page numbers, returning structured text chunks with metadata.

6.2 Indexer (indexer.py)

Manages text processing and vector storage:

- **Text Chunking:** Uses recursive character text splitting with chunk sizes between 500–1000 characters and 100–200 character overlap to keep clause context intact.
- **Embeddings:** Generates dense vector representations using Sentence-Transformers (all-MiniLM-L6-v2) or OpenAI embeddings.
- **Vector Index:** Stores vector representations in FAISS or ChromaDB along with metadata payload tags (doc_name, page_number, category).

6.3 Metadata Retriever (retriever.py)

Converts user queries into dense vectors, runs similarity searches against the index, and applies metadata filters to isolate relevant regulation categories.

6.4 Answer Generator (generator.py)

Synthesizes answers using LLM backends (Groq, Gemini, or OpenAI). If the retrieved context similarity is below the cutoff threshold, it outputs: *"The answer cannot be found in the uploaded document."*

6.5 REST API (main.py)

Built with FastAPI, exposing endpoints for querying (POST /api/query), document management (POST /api/upload), and health status checks.

6.6 Frontend (frontend/)

A single-page web interface with document upload controls, category search filters, chat interaction, and a source citations panel displaying page references.

7 Technology Stack

Layer	Technology	Purpose
Backend Framework	FastAPI 0.115+	Async REST API, OpenAPI docs
ASGI Server	Uvicorn	Async web server
PDF Parsing	PyPDF / pdfplumber	Text and metadata extraction
Vector Store	FAISS / ChromaDB	Vector similarity search
Embedding	Sentence-Transformers	Dense semantic embeddings
LLM Support	Groq / Gemini / OpenAI	Grounded language generation
Frontend	HTML5 / CSS3 / JS	Single-page web UI

8 Key Features

- **Multi-PDF Upload:** Ingest multiple regulation handbooks simultaneously.
- **Grounded Responses with Citations:** Every answer cites document name and page number.
- **Hallucination Reduction:** Temperature zero and context cutoffs prevent fabricated rules.
- **Unanswerable Detection:** Automatically flags queries missing from document context.
- **Metadata Filtering:** Filter search scope by specific policy sections.

9 Sample Queries and Responses

Query	Detected Intent	Applied Filters
"What is the minimum attendance required?"	attendance_inquiry	category = attendance
"What is the re-evaluation fee per subject?"	fee_lookup	category = examination
"What is the hostel outing timing?"	general_rules	category = hostel

10 Viva Focus Areas

- **Chunking Strategy:** Explaining why recursive character chunking with overlap prevents splitting rules mid-sentence.
- **Vector Search Mechanics:** How embeddings convert unstructured policy text into dense vectors for Cosine Similarity search.
- **Hallucination Reduction:** How distance thresholds and strict prompt constraints prevent false policy answers.
- **Metadata Filtering:** Pre-filtering vectors by document name and category to speed up retrieval and increase precision.

11 System Requirements

- Hardware: Multi-core CPU, minimum 2 GB RAM (4 GB recommended), ~500 MB storage.
- Software: Python 3.9+, pip, modern web browser.

12 Installation and Running

```
# 1. Clone / navigate to project directory
cd University_Regulation_RAG

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set API key
set GROQ_API_KEY=your_key_here

# 4. Start the server
python run.py

# 5. Open browser at:
http://localhost:8000
```

13 Results and Observations

- Index Build Time: 3–6 seconds for a 50-page regulation PDF.
- Query Latency: 100–250 ms for retrieval; 600–1200 ms for generation.
- Accuracy: Strict threshold checks reliably flag out-of-context questions.

14 Future Scope

- Multi-language support for regional language queries.
- Conversational memory for multi-turn follow-up questions.
- Role-based access control for administrative documents.

15 Conclusion

University Regulation RAG Assistant demonstrates a practical implementation of Retrieval-Augmented Generation in academic administration. By combining vector search, document chunking, metadata filtering, and grounded LLM generation, it provides reliable, source-cited answers to student policy questions.

References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
2. Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond.